Lecture 8a - Logistic Regression

We can cast a **binary classification problem** to a **regression problem**. Negative examples get $y = 0$ and positive examples get $y = 1$. We will predict positive when $h_\theta(x) > 0.5$. This means h_θ can only give output values in range [0, 1]. We can use the **sigmoid function** for this.

Overview / Recap

Training set: $\{(x^{(1)}, x^{(1)}), (x^{(2)}, y^{(2)}), \ldots, (x^{(m)}, y^{(m)})\}$

m examples: $x_0 = 0, x_i \in X, y \in [0, 1]$

$h_\theta(x) = \frac{1}{1+e^{-\theta^T x}}$

Multi-Class Problem

For a multi-class problem with i classes:

$$h_\theta^i(x) = p(y = i|x, \theta)$$

For prediction choose the i-th class for which $h_\theta^i(x)$ returns the highest value.

The only problem Logistic Regression cannot solve is highly non-linear regression problems (e.g. image recognition). The classifier to use for these problems is an ANN.

Lecture 8b - Recommender Systems

The goal is to predict if someone will like a web page, posting, movie, book etc. The old approach for this used to be a **content-based approach.** But a more recent technique is **collaborative filtering**, where we look at what similar users bought or like, and let the machine learn both the parameters and the features. Issues still remaining with collaborative filtering include how to find related products.

Collaborative Filtering

1) Initialize $x^{(1)}, \ldots, x^{(n_{dishes})}$ and $\theta^{(1)}, \ldots, \theta^{(n_{users})}$ to small random values.

2) Minimize $J(x^{(1)}, \ldots, x^{(n_{dishes})}, \theta^{(1)}, \ldots, \theta^{(n_{users})})$:

$$x_k^{(i)} := x_k^{(i)} - \alpha \left(\sum_{j:r(i,j)=1} ((\theta^{(j)})^T \theta^{(j)} - y^{(i,j)}) x_k^{(i)} + \lambda x_k^{(i)} \right)$$

$$\theta_k^{(j)} := \theta_k^{(j)} - \alpha \left(\sum_{i:r(i,j)=1} ((\theta^{(j)})^T x^{(i)} - y^{(i,j)}) x_k^{(i)} + \lambda \theta_k^{(j)} \right)$$

3) For a user with learned parameters θ and a dish with learned features x, predict a star rating as $\theta^T x$.

An issue is that for new users, collaborative filtering will always predict 0 for everything. A solution for this is **mean normalization**. For user j on dish i we predict:

$$(\theta^{(j)})^T (x^{(i)}) + \mu_i$$

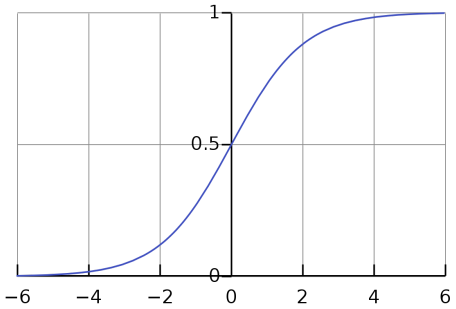
Where μ_i is the mean rating for dish i. This means that for brand new users we now rate them as the mean.

Sigmoid Function

A function in range [0, 1]:

$$g(z) = \frac{1}{1 + e^{-z}}$$

$$h_\theta(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$



Cost Function + Gradient Descent

The problem with using $h_\theta(x) = g(\theta^T x)$ in the cost function is that the cost function is not **convex** anymore. To fix this, we create a new cost function:

$$J(\theta) = -\frac{1}{m} \left(\sum_{i=1}^{m} y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_\theta(x^{(i)})) \right)$$

Note that the log here refers to the natural logarithm.

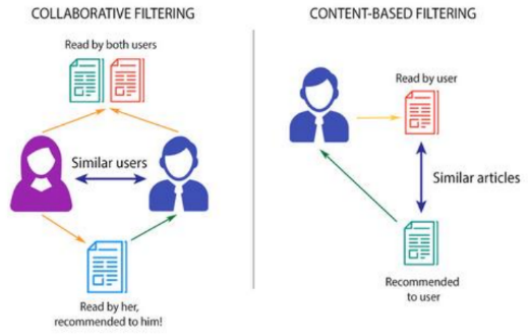
Gradient Descent works exactly the same as in linear regression, but with $h_\theta = g(\theta^T x)$.

To add **regularization** to the cost function, add the following term to $J(\theta)$:

$$+\frac{\lambda}{2m} \sum_{j=1}^{n} \theta_j^2$$

To add **regularization** to the gradient descent, add the following term to the end of the gradient descent formula:

$$+\frac{\lambda}{m} \theta_j$$



Content-Based Approach

Let $\theta^{(j)}$ be the parameter vector for user j, and $x^{(i)}$ be the feature vector for dish i. We can predict the rating of user j for dish i using:

$$(\theta^{(j)})^T (x^{(i)})$$

Optimization Algorithm:

$$\frac{1}{2} \sum_{j=1}^{n_{users}} \sum_{i:r(i,j)=1} \left((\theta^{(j)})^T x^{(i)} - y^{(i,j)} \right)^2 + \frac{\lambda}{2} \sum_{j=1}^{n_{users}} \sum_{k=1}^{n} (\theta_k^{(j)})^2$$

Where $i : r(i, j) = 1$ represents the dishes that have been rated by user j, $y^{(i,j)}$ represents the rating given to dish i by user j, and the last term represents the regularization.

The Gradient Descent update looks like:

$$\theta_k^{(j)} := \theta_k^{(j)} - \alpha \left(\sum_{i:r(i,j)=1} ((\theta^{(j)})^T x^{(i)} - y^{(i,j)}) x_k^{(i)} + \lambda \theta_k^{(j)} \right)$$

Note that for $k = 0$ the last $\lambda \theta_k^{(j)}$ term is dropped.